

**TF****TERRAFLY**

TerraFly Zero-to-Ownership Bootcamp

A minimal verified learning path, six role tracks, data/training plan, oral exam and judge-readiness gate

Verified against the packaged TerraFly project and live application on 2026-08-28.

Scientific position

Measured mountains come from a named DEM. Photo AI begins as relative shape. AI-derived metric files exist only after independent validation passes. Good-looking 3D is never substituted for evidence.

Contents

Contents	2
TerraFly Zero-to-Ownership Bootcamp	3
1. Graduation standard	3
2. Compulsory shared foundation	3
3. Minimal verified resource path	4
4. Role tracks and genuine deliverables	9
5. Staged real-data and training plan	10
6. Absolute prohibitions	12
7. Seven-day schedule	12
8. Emergency two-day schedule	12
9. Daily teach-back exercises	13
10. Fifty-question oral examination with answer key	13
11. Final judge-readiness test	15
TerraFly learning resource verification table	17
Verification notes	19
TerraFly claims ledger	20
Visual evidence and editable diagrams	22
TerraFly runtime architecture	22
Scientific states require new evidence	22
DEM Terrain workflow	23
Photo AI workflow	23
Metric calibration gate	24
Height pixels to textured mesh and measurement	24
Five auditable request flows	25

TerraFly Zero-to-Ownership Bootcamp

Audience: six beginners who must operate, trace, defend and eventually recreate the actual TerraFly project.

Companion: `TerraFly_Source_to_System_Atlas.md` Rule: watching is not completion. Every resource has a TerraFly exercise and three questions.

1. Graduation standard

A teammate owns TerraFly only when they can:

1. Run both workflows and explain the four state labels.
2. Trace a browser action to the API route, worker, artifact and test.
3. Distinguish source DEM values, raw model output, canonical relative surface, display cleanup and calibrated metres.
4. Explain every calibration gate and why rejection is correct behavior.
5. Demonstrate tests, hashes, manifests and limitations without hiding AI assistance.
6. Refuse an unsupported claim even when it would make the demo sound stronger.

Ownership statement "Yes, we used AI-assisted development. We can trace the complete request flow, explain every scientific gate, run the tests, identify which model is real versus test-only, and show why metric files are absent until independent validation passes. We own the decisions and evidence."

Why it is true:

- Complete request flows are mapped through `frontend/src/App.tsx:41-501`, `api.ts:3-73`, `backend/terrafly/main.py:49-219`, the workers and `jobs.py:28-63`.
- Scientific gates are executable rules in `calibration.py:136-163` and pass-only writing is at 276-345; `test_calibration.py:68-241` proves pass and rejection.
- Tests are not a slogan: the audited package passed 40 backend and 10 frontend tests.
- The real adapter is `depth_anything_v2.py:11-158`; the deterministic test adapter is gated by `factory.py:9-23` and tested at `test_adapter_contract.py:57-65`.
- Metric files are conditional in `calibration.py:276-345`; poor evidence remains rejected in `test_calibration.py:162-184`.
- The team owns decisions only after it can reproduce these demonstrations and explain the limitations in `CLAIMS_LEDGER.md`.

2. Compulsory shared foundation

All six members complete these before role specialization:

1. R01 or R02 for Python basics, R03 for arrays, R05 for neural networks.
2. R12-R20 for remote sensing, DEM/DSM, raster, CRS, GeoTIFF and datum.
3. R21-R28 for API, React/TypeScript, mesh and glTF concepts.
4. R29-R33 for calibration, validation and metrics.
5. R36-R40 for licensing, tests and evidence.

6. Read `README.md`, `docs/OPERATOR_GUIDE.md`, `docs/JUDGE_QA.md`, the Atlas sections 5-12 and `CLAIMS_LEDGER.md`.
7. Run DEM Terrain, Photo AI and the synthetic calibration oracle.
8. Give a five-minute teach-back without slides.

3. Minimal verified resource path

The exact titles and URLs were checked on 2026-08-28. Roles: ALL, TL team leader, AI Om, RS Sister/Saman research, QA Parth judge Q&A, GEO Member 5, EV Member 6, 3D frontend/3D owner (shared by TL plus the strongest UI learner).

R01 - Python course

[CS50's Introduction to Programming with Python](#) - Harvard CS50. Why: beginner route to functions, branches, loops, exceptions, libraries, tests and files used throughout the backend. Who: ALL; AI/EV must finish. Required: Weeks 0-6. Time: about 8 h at accelerated pace. Exercise: open `backend/terrafly/evaluation.py:9-50`; write down every variable's type and follow one test at `test_evaluation.py:10-22`. Gate questions: (1) What is a function argument? (2) Why catch a specific exception? (3) What does a test assertion prove and not prove?

R02 - Python reference

[The Python Tutorial](#) - Python Software Foundation. Why: the authoritative reference for syntax, modules, I/O, errors and environments. Who: ALL as reference; TL/AI/EV required. Required: sections 3, 4, 6, 7, 8, 12. Time: 3 h. Exercise: annotate imports and exception paths in `pipeline.py:18-96`. Questions: (1) Module versus package? (2) Why use `with` for files? (3) How does a virtual environment protect repeatability?

R03 - NumPy

[NumPy: the absolute basics for beginners](#) - NumPy. Why: every TerraFly surface is an array. Who: ALL; AI/GEO/EV required. Required: shape, ndim, size, dtype, indexing, masks, broadcasting. Time: 2 h. Exercise: load a demo `.npy`, report shape/dtype/min/max and count finite values without editing it. Questions: (1) Why float32? (2) What is the difference between shape `(h,w)` and `(w,h)`? (3) Why must NoData be masked before metrics?

R04 - Image processing

[Image Processing in OpenCV](#) - OpenCV. Why: explains channels, interpolation, smoothing and gradients behind preprocessing/display cleanup. Who: AI/GEO/3D. Required: color conversion, geometric transforms, smoothing, gradients. Time: 2 h. Exercise: compare those concepts with `imaging.py:62-73,140-175` and `artifacts.py:299-519`; identify which cleanup is display-only. Questions: (1) Why can smoothing erase roof edges? (2) What does bilinear interpolation do? (3) Why must RGB-guided cleanup not touch `relative_surface.npy`?

R05 - Neural-network intuition

[But what is a neural network? Deep learning chapter 1](#) - 3Blue1Brown. Why: gives non-mathematical intuition for learned weights and layered representation. Who: ALL. Required: whole video. Time: 19 min. Exercise: draw input image -> DAV2 -> raw field -> TerraFly conversion, labelling learned versus hand-written steps. Questions: (1) What is learned? (2) Is a high-confidence-looking picture proof? (3) Why can domain shift break a trained model?

R06 - PyTorch basics

[Learn the Basics](#) - PyTorch. Why: tensors, datasets, model, gradients, optimization and checkpoints separate inference from training. Who: AI/TL/EV. Required: tensors, DataLoaders, build model, autograd, optimization, save/load. Time: 4 h. Exercise: point to the inference-only context and device movement in `depth_anything_v2.py:74-158`; explain why TerraFly does not update weights. Questions: (1) Tensor versus NumPy array? (2) What changes during training? (3) Why disable gradients for inference?

R07 - CUDA and mixed precision

[Automatic Mixed Precision](#) - PyTorch. Why: TerraFly uses CUDA autocast; future 8 GB training needs `autocast` and `GradScaler`. Who: AI/TL. Required: `autocast`, `GradScaler`, memory/timing. Time: 45 min. Exercise: trace device selection/OOM retry at `depth_anything_v2.py:74-158`; record the device from a job manifest. Questions: (1) Why can mixed precision save memory? (2) Why is `GradScaler` training-specific? (3) What must be logged after CPU fallback?

R08 - Monocular depth with Transformers

[Monocular depth estimation](#) - Hugging Face. Why: defines the task and the Transformers inference/fine-tuning pattern. Who: AI/TL/QA. Required: task, inference, fine-tuning overview, evaluation. Time: 1.5 h. Exercise: compare the tutorial output with TerraFly's Prediction contract in `inference/base.py:9-19`. Questions: (1) Relative versus metric depth? (2) Why is scale ambiguous? (3) Which metadata must TerraFly add around the model?

R09 - Depth Anything V2 project

[Depth Anything V2](#) - model authors. Why: authoritative overview of the current baseline and its relative/metric variants. Who: AI/TL/QA. Required: abstract, model strategy, relative versus metric results. Time: 45 min. Exercise: explain why TerraFly uses a relative Large checkpoint and its own evidence gate. Questions: (1) What training data strategy is described? (2) Are all checkpoints metric? (3) Why does a benchmark paper not prove TerraFly scene accuracy?

R10 - Depth Anything V2 code

[Depth Anything V2 repository](#) - model authors. Why: usage, model sizes and license context. Who: AI/TL/EV. Required: README usage, model table and license notes. Time: 1 h. Exercise: map upstream model loading to `depth_anything_v2.py:11-73`. Questions: (1) Which checkpoint is configured? (2) What is the difference between upstream code and HF wrapper? (3) Why record a revision rather than only a model name?

R11 - Exact model card

[Depth-Anything-V2-Large-hf model card](#) - Hugging Face/model authors. Why: exact checkpoint, intended use and license for TerraFly. Who: ALL read; AI/RS/EV own. Required: details, usage and license. Time: 30 min. Exercise: find the model name/revision in a verified `job_manifest.json` and compare with `config.py:21-29`. Questions: (1) What license applies? (2) Why is commercial readiness blocked? (3) Is it trained specifically for nadir satellite nDSM?

R12 - Remote sensing fundamentals video

[Overview of Webinar Series, ARSET, and an Introduction to Satellite Remote Sensing](#) - NASA ARSET. Why: active/passive sensors, orbits and spatial/spectral/radiometric/temporal resolution. Who: ALL; GEO/RS required. Required: sensor and four-resolution sections. Time: 72 min. Exercise: create a one-page table describing the bundled optical and DEM files using all four resolution concepts. Questions: (1) Spatial versus spectral resolution? (2) Why can two acquisitions disagree temporally? (3) Is a 10 m display grid automatically 10 m elevation evidence?

R13 - Elevation models

[Introduction to Digital Elevation Models: 2025 Spring Webinar Series](#) - OpenTopography. Why: distinguishes elevation surface types, interpolation and source resolution. Who: ALL; GEO/QA required. Required: DEM/DTM/DSM and global/high-resolution product sections. Time: 75 min. Exercise: classify `metric_surface.tif` separately for DEM Terrain and Photo AI calibration. Questions: (1) DEM, DSM and DTM difference? (2) What is nDSM? (3) Why can forest canopy alter a surface model?

R14 - GIS, raster and CRS

[A Gentle Introduction to GIS](#) - QGIS Project. Why: foundation for raster cells, georeferencing and coordinate systems. Who: ALL; GEO owns. Required: Raster Data and Coordinate Reference Systems chapters. Time: 3 h. Exercise: explain every field returned by `imaging.py:76-137` for a demo GeoTIFF. Questions: (1) Vector versus raster? (2) Geographic versus projected CRS? (3) Why is EPSG code alone not a vertical datum?

R15 - Copernicus Browser

[About the Browser](#) - Copernicus Data Space Ecosystem. Why: official search, AOI, visualization and analytical download workflow. Who: GEO/RS/TL. Required: product search, visualize, AOI and image download. Time: 1 h. Exercise: locate one cloud-light mountain scene and complete the Stage A metadata sheet before downloading. Questions: (1) Visualization versus analytical download? (2) Why record product ID/date? (3) How will you make the DEM cover the same AOI?

R16 - Sentinel-2 bands

[Sentinel-2 L2A](#) - Copernicus Data Space Ecosystem. Why: band names/resolutions and processed surface-reflectance context. Who: GEO/RS. Required: band table and metadata. Time: 30 min. Exercise: write why B02, B03 and B04 can share a 10 m true-color grid. Questions: (1) Which band is red? (2) What is L2A? (3) Why not silently mix 10 m and 20 m bands?

R17 - Sentinel-2 true color

[Sentinel-2 L2A examples](#) - Copernicus. Why: verifies the true-color order [B04, B03, B02]. Who: GEO/RS/3D. Required: True Color example. Time: 20 min. Exercise: inspect a downloaded raster in QGIS and verify the rendered red/green/blue band order. Questions: (1) Why is B04 first in RGB? (2) What does contrast stretching change? (3) Should a colored export be used as numeric elevation truth?

R18 - NASA SRTM

[Shuttle Radar Topography Mission \(SRTM\)](#) - NASA Earthdata. Why: provenance and limits of a common global DEM. Who: GEO/RS/QA. Required: mission, coverage and product overview. Time: 45 min. Exercise: record product/source, native spacing, vertical reference, tile ID and terms for Stage A. Questions: (1) Is SRTM optical imagery? (2) What does native resolution mean after resampling? (3) Why must datum be recorded?

R19 - GeoTIFF with Rasterio

[Python Quickstart](#) - Rasterio. Why: same library used to inspect/read/write TerraFly rasters. Who: GEO/AI/EV. Required: open, metadata, bands, bounds, transform, CRS, reading/writing. Time: 1.5 h. Exercise: inspect both Stage A files and compare their bounds/CRS/resolution/NoData. Questions: (1) What does the affine transform encode? (2) Why are bounds needed? (3) How is a mask different from a numeric zero?

R20 - Vertical datums

[NOAA/NOS's VDatum: A tutorial on datums](#) - NOAA. Why: prevents the dangerous belief that all metre elevations share the same zero. Who: GEO/TL/QA. Required: horizontal and vertical datum concepts. Time: 1 h. Exercise:

locate where TerraFly records the user's datum label in `terrain.py:213-348` and the alignment report. Questions: (1) Ellipsoid versus geoid height? (2) Can two metre rasters differ by an offset? (3) Why is an EPSG horizontal CRS insufficient?

R21 - FastAPI

[Tutorial - User Guide](#) - FastAPI. Why: explains TerraFly routes, typed requests, errors, background work, static files and tests. Who: TL/AI/EV. Required: First Steps, Request Body, Errors, Background Tasks, Static Files, Testing. Time: 3 h. Exercise: trace `POST /api/jobs` at `main.py:49-67`, including status code and worker. Questions: (1) Route versus worker? (2) Why return a job before inference finishes? (3) What converts a Pydantic model to JSON?

R22 - File uploads

[Request Files](#) - FastAPI. Why: multipart upload contract used by both workflows. Who: AI/EV/3D. Required: `UploadFile`, file bytes and form data. Time: 45 min. Exercise: match `api.ts:3-69` `FormData` fields to `main.py:49-136` parameters. Questions: (1) Why multipart? (2) Why stream/chunk before decode? (3) Where is the 25 MB gate?

R23 - Pydantic models

[Models](#) - Pydantic. Why: TerraFly job/state/calibration schema is executable documentation. Who: TL/AI/EV. Required: fields, validation and serialization. Time: 1 h. Exercise: explain `JobManifest` at `schemas.py:23-48` and find its TypeScript counterpart. Questions: (1) Why enum scientific states? (2) What happens to invalid field data? (3) Why keep backend and frontend types aligned?

R24 - React

[Quick Start](#) - React. Why: state, events, conditional rendering and lists drive the workflow UI. Who: 3D/TL; others skim. Required: components, state, events, lists, conditions. Time: 2 h. Exercise: trace `workflow`, `job`, `busy` and polling state in `App.tsx:41-153`. Questions: (1) Props versus state? (2) Why cleanup a polling timer? (3) How does conditional rendering enforce scientific state?

R25 - TypeScript

[TypeScript for the New Programmer](#) - Microsoft. Why: compile-time checks connect UI code to the API contract. Who: 3D/TL/EV. Required: JS-to-TS mental model and types. Time: 1 h. Exercise: select one field in `types.ts` and trace every consumer. Questions: (1) Compile-time versus runtime validation? (2) Why can an API still return unexpected data? (3) What does an optional field mean?

R26 - Three.js fundamentals

[Fundamentals](#) - Three.js. Why: scene, camera, renderer, geometry, material and lights are the viewer. Who: 3D/TL/QA. Required: scene/camera/renderer/mesh/material/light. Time: 1.5 h. Exercise: label those objects in `SurfaceViewer.tsx:112-260`. Questions: (1) Geometry versus material? (2) Why is a camera needed? (3) Does brighter lighting mean greater elevation?

R27 - Mesh attributes

[Custom BufferGeometry](#) - Three.js. Why: positions, indices, normals and UVs are TerraFly's mesh contract. Who: 3D/AI. Required: indexed geometry and attributes. Time: 1 h. Exercise: draw one raster cell as four vertices and two triangles, then compare `surfaceGeometry.ts:202-255`. Questions: (1) What is an index triple? (2) What does a normal control? (3) What do UV coordinates reference?

R28 - glTF/GLB

[glTF Tutorial](#) - Khronos Group. Why: explains the portable binary artifact TerraFly exports. Who: 3D/EV/TL. Required: scenes/nodes, buffers, accessors, meshes, materials, textures. Time: 2 h. Exercise: match each GLB element asserted by `test_artifacts.py:31-128` to `artifacts.py:112-296`. Questions: (1) glTF versus GLB? (2) What is an accessor? (3) Why embed texture for portability?

R29 - Linear regression

[Linear Regression. Clearly Explained!!!](#) - StatQuest. Why: TerraFly fits $\text{elevation} = \text{scale} \times \text{relative} + \text{offset}$. Who: ALL; AI/GEO/EV own. Required: 0:37-14:15; optional 25:26. Time: 28 min. Exercise: interpret scale 40 and offset 100 in the oracle report. Questions: (1) What are slope and intercept? (2) Why require positive slope? (3) Why can a line fit still have local residuals?

R30 - R-squared

[R-squared. Clearly Explained!!!](#) - StatQuest. Why: $R^2 \geq 0.50$ is one TerraFly calibration gate. Who: ALL. Required: whole video. Time: 11 min. Exercise: find R^2 in `calibration_report.json` and the threshold in `calibration.py:136-163`. Questions: (1) What does R^2 measure? (2) Can good R^2 coexist with large metre error? (3) Why require RMSE too?

R31 - Train/validation/test separation

[Cross-validation: evaluating estimator performance](#) - scikit-learn. Why: testing on fitting data creates false confidence. Who: ALL; AI/GEO/EV required. Required: holdout, validation vs test, CV. Time: 1 h. Exercise: explain the calibration checkerboard and why future tiles must split by city first. Questions: (1) Training versus validation versus test? (2) Why keep final test untouched? (3) Why is the oracle not a real test set?

R32 - Leakage

[Common pitfalls and recommended practices](#) - scikit-learn. Why: overlapping geographic tiles and preprocessing can leak information. Who: AI/GEO/EV/TL. Required: inconsistent preprocessing, data leakage, randomness. Time: 1 h. Exercise: design a city-level split manifest before tiling DFC23. Questions: (1) How do overlapping tiles leak? (2) When must normalization statistics be fit? (3) Why record random seeds and parent scenes?

R33 - Height metrics

[Metrics and scoring: quantifying the quality of predictions](#) - scikit-learn. Why: RMSE, MAE and R^2 are required evaluation vocabulary. Who: ALL; EV/AI/QA own. Required: regression metrics. Time: 1 h. Exercise: hand-check the example in `test_evaluation.py:10-22`. Questions: (1) Why does RMSE emphasize large errors? (2) What does bias reveal? (3) Why report per-building and worst-case results as well as averages?

R34 - DFC23

[2023 IEEE GRSS Data Fusion Contest](#) - IEEE GRSS. Why: Track 2 directly covers optical/SAR, building masks and nDSM across 17 cities. Who: AI/GEO/RS/EV/TL. Required: Track 2 description, terms, citation/acknowledgement. Time: 1 h before download. Exercise: create a signed data-gate checklist; do not download or train until all terms and pairing fields are recorded. Questions: (1) Which labels exist? (2) Why split by city? (3) What permission/acknowledgement is required?

R35 - Model cards

[Model Cards](#) - Hugging Face. Why: future TerraFly checkpoints need scope, data, metrics, limitations and license. Who: AI/EV/RS/TL. Required: metadata, intended uses, limitations, evaluation. Time: 45 min. Exercise: critique docs/MODEL_METRIC_CARD.md:1-27; explain why BLOCKED is scientifically valid. Questions: (1) What must a model card disclose? (2) Why version a new overhead checkpoint separately? (3) What results must be held-out?

R36 - Google Geo Guidelines

[Geo Guidelines](#) - Google. Why: governs attribution and restricts reconstructing 3D from captured Google Earth output. Who: ALL; RS/QA/EV own. Required: Google Earth and attribution/use restrictions. Time: 30 min. Exercise: remove Google Earth screenshots from the data plan; write an allowed comparison caption with attribution only if applicable. Questions: (1) Can captured Earth output train/reconstruct TerraFly? (2) What attribution is required? (3) What source should replace it?

R37 - Creative Commons licenses

[About CC Licenses](#) - Creative Commons. Why: BY, SA, NC and ND conditions affect imagery, models and presentations. Who: ALL; RS/EV own. Required: license elements and six license types. Time: 30 min. Exercise: explain the NC consequence of the DAV2 Large checkpoint in THIRD_PARTY_NOTICES.md. Questions: (1) What does BY require? (2) What does NC restrict? (3) Does a public download mean unrestricted reuse?

R38 - pytest

[Get Started](#) - pytest. Why: backend evidence uses readable test functions and assertions. Who: AI/EV/TL. Required: discovery, assertions, exceptions, temporary paths. Time: 1 h. Exercise: run one named test and explain arrange/action/assert in it. Questions: (1) How does pytest discover tests? (2) Why use tmp_path? (3) Does a passing synthetic test prove real accuracy?

R39 - Vitest

[Getting Started](#) - Vitest. Why: React and geometry contracts are tested with the Vite toolchain. Who: 3D/EV/TL. Required: writing tests and vitest run. Time: 45 min. Exercise: trace one assertion in SurfaceViewer.test.ts:15-142 to a pure function. Questions: (1) Unit versus browser test? (2) Why test geometry outside WebGL? (3) What remains after 10 tests pass?

R40 - SHA-256

[Secure Hash Standard](#) - NIST. Why: manifests bind results to exact bytes. Who: ALL; EV owns. Required: digest purpose and change detection. Time: 30 min. Exercise: recompute a job artifact hash and compare it with job_manifest.json. Questions: (1) What changes a digest? (2) Why can the manifest not hash itself straightforwardly? (3) Why does a matching hash not prove accuracy?

4. Role tracks and genuine deliverables

Names describe provisional responsibility, not completed contribution. A person may claim a role only after delivering and teaching the listed evidence.

4.1 Team leader - integration and scientific contract

Study R01-R03, R05-R11, R13-R14, R20-R35, R38-R40. Trace all five request flows. Deliver: a hand-drawn state/evidence diagram; live two-workflow demo; one rejected calibration demo; final claim review; setup/backup checklist. Must answer oral questions 1-50 at 80% and all red-line questions 1, 5, 10, 18, 26, 35, 40, 47, 49, 50 correctly.

4.2 Om - backend, AI, CUDA, tiling and model limitations

Study R01-R11, R19, R21-R23, R29-R35, R38. Deliver: line-by-line teach-back of `pipeline.py`, inference package and tiling tests; real-model revision/device evidence; CPU fallback explanation; one tiled smoke run; proposal for an overhead multi-task baseline. Must explicitly distinguish inference, calibration and training.

4.3 Sister/Saman - research, narrative, diagrams and citations

Study R05, R09-R20, R34-R37. Deliver: official problem trace; one-page alternatives/gap matrix; data-source/license records; editable diagrams; all slide claims linked to claims ledger. Must never fabricate a source, result, contribution or timestamp.

4.4 Parth - judge Q&A, feasibility, scalability and impact

Study R05, R09-R14, R18, R20, R26-R37. Deliver: a 6-minute demo script; 20 hostile judge questions; engineering/scientific/deployment feasibility split; demonstrated/potential/unproved impact slide; two rehearsed limitation answers without defensiveness.

4.5 Member 5 - geospatial data and evaluation

Study R03-R04, R12-R20, R29-R34. Deliver: Stage A Sentinel-2/SRTM pair; QGIS inspection screenshots; metadata/license sheet; CRS/bounds/resolution/NoData audit; alignment report explanation; geographic split manifest design; residual/worst-case analysis template.

4.6 Member 6 - tests, artifacts, evidence, licenses and backup demo

Study R01-R03, R11, R19-R23, R28-R40. Deliver: run logs for 40+10 tests; artifact hash recheck; pass and rejection artifact inventories; license checklist; backup offline results/video; final reproducibility packet. Must explain what each artifact is not.

4.7 Frontend/3D ownership assignment

Because the provisional list does not name a dedicated UI person, the team leader must assign this subtrack to the strongest React learner (possibly shared with Om or Member 6). Complete R24-R28 and deliver a viewer teach-back covering mesh attributes, cleanup, raycasting, A/B sampling, modes, first-person limitations and WebGL cleanup.

5. Staged real-data and training plan

Stage A - one real Sentinel-2 + SRTM mountain demonstration

Goal: prove real data ingestion and measured DEM visualization, not AI accuracy.

1. In [Copernicus Browser](#), sign in if required, draw a compact mountain AOI with low cloud, select Sentinel-2 L2A, inspect acquisition date/cloud, and record product ID, date, AOI, provider, terms and attribution.
2. Use analytical download where possible. Build true color in B04 red, B03 green, B02 blue. Prefer a GeoTIFF rather than a screenshot or styled JPEG.
3. Obtain the matching NASA SRTM tile through an authorized NASA Earthdata/distributor path. Record product/tile, native resolution, acquisition lineage, vertical reference, void/NoData value, terms and hash.
4. Open both in QGIS. Inspect layer properties: CRS/EPSG, extent/bounds, width/height, band count/order, pixel size, data type, NoData/mask, units and statistics.

5. Reproject to a suitable projected metre CRS for the AOI. Clip both to the same overlap. Keep the DEM at its honest native information; do not claim that resampling creates detail.
6. Export optical RGB GeoTIFF and single-band metre DEM. Keep original downloads and a processing log. Do not overwrite sources.
7. Run TerraFly DEM Terrain, enter the exact source and datum labels, inspect alignment coverage and NoData, then verify `metric_surface.tif`, report and hashes.
8. Capture one clean screenshot and a provenance slide. Say: "The DEM controls geometry and metre values; Sentinel-2 controls color."

Acceptance: overlap is correct, source/datum visible, no edge shift, no unexplained stripes, no Google Earth data, hashes recorded, and a second member can repeat it.

Stage B - small paired RGB + DSM/nDSM evaluation set

Acquire several licensed, co-registered scenes with same-scene optical input, independent DSM/nDSM truth, masks, CRS/affine, metre units and vertical datum. Freeze TerraFly/model revision. Do no training. Run the unchanged baseline and report RMSE, MAE, bias, correlation/R2, P95 and residual maps by scene/land cover/building. Inspect worst cases before averages.

Stage C - licensed multi-city training data

Evaluate DFC23 Track 2 subject to its terms: 17-city optical/SAR inputs, building annotations and nDSM references. Record acceptance/citation, file inventory, pair manifest, city IDs, missing pairs, duplicates, NoData, units, datum and distributions. Start RGB-only; treat SAR as a later ablation.

Stage D - unchanged baseline evaluation

Before tuning, lock code, model revision, preprocessing and geographic test cities. Run DAV2 plus simple calibration and a compact frozen-encoder multi-task baseline. Publish configuration, dataset hashes, per-city metrics, residual maps and failure gallery. This prevents crediting training for an unmeasured change.

Stage E - train only after the gate

Model: pretrained compact overhead-capable encoder; building-mask decoder; height-above-local-ground decoder; uncertainty/log-variance decoder. Loss: Dice+BCE mask, masked Huber/L1 height, boundary gradient loss and uncertainty loss. Weight buildings/roof edges. Augment 90-degree rotations, flips and bounded radiometric/atmospheric changes; never break cross-modal alignment.

RTX 5060 8 GB start: 384 px tiles; try 512 only after profiling; mixed precision; batch 1-2; gradient accumulation to effective 8-16; freeze encoder first; progressively unfreeze with smaller encoder learning rate; deterministic seed; checkpoint best geographic validation score; early stop; retain optimizer/config/environment/dataset hashes. Split entire cities before tiling. Never let overlapping windows from one parent scene cross splits.

Evaluate untouched cities: footprint IoU/F1; pixel and per-building height MAE/RMSE/bias/median/P95; edge score; error by city, height, roof, sensor and land cover; uncertainty coverage; residual maps; worst 20 cases. Create a model card with intended use, exclusions, data, split, metrics, limitations, license and checkpoint hash.

Stage F - separately versioned TerraFly integration

Add a new adapter/model ID without silently replacing DAV2. Preserve old benchmarks, migration/config choice and all raw outputs. Run backend/frontend/security/artifact tests plus real held-out regression. Update licenses, manifest schema and claims ledger. A failed comparison means do not promote.

6. Absolute prohibitions

- Do not train on TerraFly's own predictions. - Do not use unrelated optical and DEM files as paired training examples. - Do not use colored preview PNGs as ground truth. - Do not use SAC thermal colorization arrays as height labels. - Do not quote synthetic calibration error as model accuracy. - Do not claim disaster prediction, survey-grade accuracy or operational deployment without evidence. - Do not use Google Earth captured output to reconstruct or train the 3D system.

The user's independent RGB.byte.tif and Mumbai SRTM file are ingestion examples unless they are the same scene, intentionally co-registered and licensed for the purpose. Filenames and city labels do not prove a pair.

7. Seven-day schedule

Day	Shared work	Role work	Teach-back / acceptance
1 - Truth and operation	Read Atlas 1-7; R05, R12-R14; run both workflows	TL state diagram; RS problem/gap; QA claim matrix	Every member explains relative vs georeferenced vs metric and runs DEM Terrain.
2 - Code path	R01/R02, R21-R23; trace flows A-D	AI backend; EV tests/artifacts; 3D API types	Each member traces one request with file+line and one test.
3 - Arrays, model and 3D	R03-R11, R24-R28	AI direction/tiling; 3D mesh; EV GLB contract	Build a pixel-to-mesh diagram; explain roof-as-hole repair.
4 - GIS and real data	R15-R20	GEO prepares Stage A; RS licenses	Present QGIS metadata table and matching AOI evidence before running.
5 - Calibration and evaluation	R29-R35	AI/EV pass+reject; QA metric answers	Demonstrate rejection, oracle pass, blocked real metric card.
6 - Testing and judge defense	R36-R40; run full checks	QA mock panel; EV backup; TL feasibility	Two hostile mock panels; every false claim corrected immediately.
7 - Ownership exam and final demo	Review Atlas/ledger	Finish role deliverables	50-question oral exam, 80% pass, red-line 100%, two clean demo rehearsals.

8. Emergency two-day schedule

Day 1 - operate and trace (10-12 h)

- 0:00-1:00: Atlas sections 1-7 and the claims ledger.
- 1:00-2:30: run DEM Terrain, Photo AI and calibration oracle.
- 2:30-4:00: R03, R05, R13-R14 essentials.
- 4:00-6:00: trace flows A-C in code and matching tests.
- 6:00-7:30: R26-R30 essentials; mesh and calibration teach-back.
- 7:30-9:00: role assignments and evidence collection.
- 9:00-10:00: each member gives the 30-second truth plus one limitation.

Day 2 - defend and repeat (10-12 h)

- 0:00-2:00: Stage A data/metadata rehearsal or verified bundled fallback.
- 2:00-3:30: full tests, build, hashes and backup.

- 3:30-5:00: R31-R40 essentials and prohibited claims.
- 5:00-7:00: oral exam in pairs; fix weak answers from source lines.
- 7:00-9:00: two complete judge demos, including one forced rejection.
- 9:00-10:00: final readiness test and role-deliverable sign-off.

9. Daily teach-back exercises

1. State cards: teammate draws evidence transitions; another tries to trick them with a georeferenced-but-relative GeoTIFF.
2. Request relay: six members each explain one hop from button to artifact, then swap roles.
3. Array clinic: identify shape, dtype, units and consumer for five artifact files.
4. GIS defense: show CRS, transform, bounds, resolution, NoData, source and datum for a real pair.
5. Gate court: one member prosecutes a calibration pass; another argues why it should reject using report fields.
6. Failure gallery: explain roof hole, tile seam, stretched walls, global tilt and misregistration without claiming all are solved.
7. Judge panel: answer in 30 seconds, then point to one source line and one test/artifact.

10. Fifty-question oral examination with answer key

Passing: 40/50 plus every RED LINE question correct. Answers must be spoken in plain language before the source reference is opened.

1. RED LINE - What is TerraFly? A two-path local workbench: DEM Terrain visualizes supplied metric elevation; Photo AI estimates relative shape and only exports AI-derived metres after independent calibration. [schemas.py:9-13; main.py:49-204]
2. Why two workflows? They preserve different evidence origins and prevent supplied DEM values from being confused with AI inference. [docs/JUDGE_QA.md:7-17]
3. What is Relative? Canonical 0-to-1 vertical ordering with no metres/vertical datum. [conventions.py:29-98]
4. What is Georeferenced Relative? Relative height plus horizontal CRS/transform/bounds, still no vertical scale. [schemas.py:9-13; test_geotiff.py:32-86]
5. RED LINE - When may Photo AI create metric files? Only after independent controls/validation pass all gates. [calibration.py:136-163,276-345]
6. Metric Source DEM versus Metric Calibrated? The first inherits a source DEM; the second applies a validated affine transform to AI relative output. [terrain.py:213-348; calibration.py:276-345]
7. Why can a PNG not provide metres? It lacks horizontal scale/CRS and vertical scale, offset and datum. [imaging.py:140-175; docs/JUDGE_QA.md:27-33]
8. Can a GeoTIFF still be vertically relative? Yes; georeferencing usually establishes horizontal mapping only. [test_geotiff.py:32-86]
9. What controls mountain geometry? In DEM Terrain, the source DEM; optical image controls texture. [terrain.py:84-192]
10. RED LINE - Does resampling a 30 m DEM to a 10 m grid create 10 m evidence? No. It aligns grids and preserves native information limits. [terrain.py:84-145; test_terrain.py:65-137]
11. What is NoData? Invalid/missing cells that must be masked, not treated as elevation zero. [terrain.py:34-77; test_calibration.py:219-241]

12. What does an affine raster transform encode? Pixel-to-map origin, pixel sizes and rotation/shear. [imaging.py:76-137]
13. CRS versus vertical datum? CRS maps horizontal coordinates; vertical datum defines the zero/reference for elevation. [docs/REAL_TERRAIN_DATA_GUIDE.md:24-51]
14. Which real model runs? depth-anything/Depth-Anything-V2-Large-hf, revision recorded per job. [config.py:21-29; depth_anything_v2.py:11-73]
15. What is the deterministic adapter? A predictable, explicitly gated test-only substitute. [deterministic.py:9-41; factory.py:9-23]
16. Inference versus training? Inference applies frozen weights; training updates them using labelled data/loss. TerraFly currently performs inference. [depth_anything_v2.py:74-158]
17. What is raw_model_output.npy? Untouched model field before direction/normalization. [pipeline.py:44-76; artifacts.py:735-888]
18. RED LINE - Why did roofs become holes historically? The inverse-depth/proximity output was inverted as if it were ordinary depth. [conventions.py:9-98; test_adapter_contract.py:24-55]
19. How is recurrence prevented? Explicit convention plus synthetic raised-roof NumPy/GLB regressions. [test_adapter_contract.py:24-55; test_artifacts.py:130-166]
20. What is robust normalization? Map global 2nd-98th percentile range to 0-1 and clamp outliers. [conventions.py:29-98]
21. Why tile? Bound memory while retaining more local detail than shrinking the whole image. [tiling.py:8-99]
22. Why align tile scale/offset? Monocular crop outputs can have arbitrary local affine differences. [tiling.py:41-99; test_tiling.py:43-64]
23. What does feather blending do? Reduces hard seams by weighting overlaps smoothly. [tiling.py:18-99]
24. Does tiling prove better accuracy? No; it is an engineering strategy that still needs evaluation. [docs/JUDGE_QA.md:99-101]
25. Canonical versus display grid? Canonical is calibration/analysis input; display grid may be cleaned for visual mesh only. [artifacts.py:299-519, 735-888]
26. RED LINE - Can the Structures layer be called detected buildings? No; it is nonsemantic candidate extrusion and can include trees or miss roofs. [artifacts.py:590-732]
27. What are mesh vertices? 3D points derived from grid locations and displayed height. [surfaceGeometry.ts:202-255]
28. What are triangle indices? Triples that connect vertices into faces. [artifacts.py:112-296]
29. What are normals? Unit directions used by lighting to shade the surface. [artifacts.py:83-98]
30. What are UVs? 2D coordinates that map the source texture onto mesh vertices. [artifacts.py:112-296]
31. Why neutral steep faces? Top-down imagery lacks real facade pixels; neutral shading avoids stretched color. [artifacts.py:112-296]
32. What does vertical exaggeration change? Only displayed vertex height, never stored arrays/GeoTIFF. [surfaceGeometry.ts:202-255; docs/JUDGE_QA.md:132-134]
33. How does an A/B click get a value? Raycast -> UV/pixel -> bilinear sample from relative or metric analysis grid. [SurfaceViewer.tsx:262-317; surfaceGeometry.ts:144-200]
34. When can horizontal distance/slope be shown? When geospatial metadata uses projected metre units. [surfaceGeometry.ts:122-142; App.tsx:155-186]
35. RED LINE - State the calibration equation and evidence split. $\text{elevation_m} = \text{scale} * \text{relative} + \text{offset}$; controls fit, separate held-out data approves/rejects. [calibration.py:54-89, 136-163]
36. Why positive scale? Prevents an upside-down height relation. [calibration.py:136-163]

37. Why relative-span gate? Nearly constant controls cannot establish a stable metre scale.
[calibration.py:136-163]
38. Why inlier-ratio and spatial-coverage gates? The fit must explain most controls distributed across both axes.
[calibration.py:121-163]
39. Why RMSE and R2 together? RMSE limits metre error; R2 requires meaningful explained variation.
[calibration.py:136-163]
40. RED LINE - What does calibration rejection do? Saves evidence/report but keeps metric .npy/GeoTIFF absent and state nonmetric. [calibration.py:276-345; test_calibration.py:162-184]
41. Why exact grid for reference DSM? Silent reprojection can hide misregistration and blur boundaries.
[calibration.py:348-461]
42. What is the bundled calibration oracle? Reference constructed from output with known affine relation/outliers to test software. [sample_data/terrafly_calibration_demo_metadata.json]
43. Why is near-zero oracle RMSE not model accuracy? Truth is derived from the prediction, not an independent landscape. [docs/MODEL_METRIC_CARD.md:1-27]
44. What does SHA-256 prove? Exact byte identity/change detection, not correctness. [artifacts.py:18-33]
45. Why have job.json and job_manifest.json? Mutable progress versus final provenance/artifact evidence.
[jobs.py:28-63; artifacts.py:735-888]
46. What do 40+10 passing tests prove? The coded contracts pass their fixtures; not all real-world accuracy/security/compatibility. [backend/tests; frontend/src/*.test.*]
47. RED LINE - May Google Earth screenshots reconstruct/train TerraFly? No; use licensed analytical optical data and a matching DEM. [docs/JUDGE_QA.md:19-25; Geo Guidelines R36]
48. Why not train on unrelated RGB and Mumbai SRTM? They are not the same aligned scene/target pair.
[docs/TRAINING_ROADMAP.md:71]
49. RED LINE - Which claims are blocked? Real Photo AI accuracy, survey grade, disaster prediction and operational readiness. [CLAIMS_LEDGER.md]
50. RED LINE - What is the best honest judge demo? Licensed matching optical+DEM mountain in DEM Terrain, then experimental Photo AI and synthetic gate clearly labelled. [docs/DEMO_SCRIPT.md:13-96]

11. Final judge-readiness test

The team is ready only if every box is true:

- [] Fresh start on the judging machine reaches the health endpoint and UI.
- [] Fast/full checks are saved; current result is 40 backend and 10 frontend tests.
- [] DEM Terrain completes with a rehearsed pair; source, datum, native resolutions and attribution are visible.
- [] Photo AI job displays exact model revision/device and `relative - not metres` before calibration.
- [] One passing oracle and one rejection are rehearsed; metric files are absent on rejection.
- [] A second person verifies one artifact SHA-256 and explains why that is not accuracy.
- [] The real model metric card is shown as BLOCKED, not replaced with oracle error.
- [] Google Earth is absent from reconstruction/training data.
- [] Every member gives the 30-second truth and answers their role's hostile questions.
- [] The technical leader can trace flows A-E without notes.
- [] Data/AI owners can explain city-first split, no overlap leakage, frozen baseline and RTX 5060 settings.

- [] The backup contains installers/cache plan, sample files, verified artifacts, screenshots, test report and a short recorded demo.
- [] Slide claims match `CLAIMS_LEDGER.md`; no survey/disaster/operational claim appears.
- [] The exact ownership statement is delivered calmly and supported with code/tests.

Stop condition: one red-line oral question wrong, one missing source/datum, one unexplained calibration file, or one unsupported accuracy statement means the team is not yet ready. Fix knowledge/evidence before changing the claim.

TerraFly learning resource verification table

Checked 2026-08-28. Official/project-author sources are preferred. Approximate study time means the required subset, not the whole site. Detailed exercises and gate questions are in the bootcamp.

ID	Exact title	Owner	URL	Required part	Time	Covers
R01	CS50's Introduction to Programming with Python	Harvard CS50	https://cs50.harvard.edu/python/	Weeks 0-6; focus functions, conditions, loops, exceptions, libraries, tests, file I/O	8 h	Python, tests, files
R02	The Python Tutorial	Python Software Foundation	https://docs.python.org/3/tutorial/	Sections 3, 4, 6, 7, 8 and 12	3 h	Syntax, modules, I/O, errors, environments
R03	NumPy: the absolute basics for beginners	NumPy	https://numpy.org/doc/stable/user/absolute_beginners.html	Arrays, shape, ndim, size, dtype, indexing, broadcasting	2 h	Arrays, shapes, dtypes
R04	Image Processing in OpenCV	OpenCV	https://docs.opencv.org/4.x/d2/d96/tutorial_py_table_of_contents_imgproc.html	Color conversion, geometric transforms, smoothing, gradients	2 h	Image preprocessing and edges
R05	But what is a neural network? Deep learning chapter 1	3Blue1Brown	https://www.youtube.com/watch?v=aircAruvnKk	Entire video	19 min	Neural-network fundamentals
R06	Learn the Basics	PyTorch	https://docs.pytorch.org/tutorials/beginner/basics/intro.html	Tensors, datasets, model, autograd, optimization, save/load	4 h	Inference vs training, tensors
R07	Automatic Mixed Precision	PyTorch	https://docs.pytorch.org/tutorials/recipes/recipes/amp_recipe.html	autocast, GradScaler, timing/memory	45 min	CUDA memory and mixed precision
R08	Monocular depth estimation	Hugging Face	https://huggingface.co/docs/transformers/tasks/monocular_depth_estimation	Task definition, inference pipeline, fine-tuning overview, evaluation	1.5 h	Transformers and monocular depth
R09	Depth Anything V2	Depth Anything authors	https://depth-anything-v2.github.io/	Abstract, method summary, relative versus metric models	45 min	Current model context
R10	Depth Anything V2 repository	Depth Anything authors	https://github.com/DepthAnything/Depth-Anything-V2	README, usage, model table, license notes	1 h	Checkpoints and limitations
R11	Depth-Anything-V2-Large-hf model card	Hugging Face / authors	https://huggingface.co/depth-anything/Depth-Anything-V2-Large-hf	Model details, intended use, license	30 min	Exact TerraFly checkpoint
R12	Introduction to Satellite Remote Sensing for Air Quality Applications, Part 1	NASA ARSET	https://www.youtube.com/watch?v=vGU4UqhHhAA	Sensors; spatial, spectral, radiometric and temporal resolution	72 min	Remote sensing fundamentals
R13	Introduction to Digital Elevation Models: 2025 Spring Webinar Series	OpenTopography	https://opentopography.org/node/3375	DEM/DTM/DSM, interpolation, global and high-resolution sources	75 min	Elevation models
R14	A Gentle Introduction to GIS	QGIS Project	https://docs.qgis.org/3.44/en/docs/gentle_gis_introduction/index.html	Raster Data chapter and Coordinate Reference Systems chapter	3 h	Raster, CRS, projected vs geographic
R15	About the Browser	Copernicus Data Space Ecosystem	https://documentation.dataspace.copernicus.eu/Applications/Browser.html	Search, visualize, AOI and analytical image download	1 h	Copernicus Browser

ID	Exact title	Owner	URL	Required part	Time	Covers
R16	Sentinel-2 L2A	Copernicus Data Space Ecosystem	https://documentation.dataspace.copernicus.eu/APIs/SentinelHub/Data/S2L2A.html	Band table; B02/B03/B04 are 10 m	30 min	Optical satellite data
R17	Sentinel-2 L2A examples	Copernicus Data Space Ecosystem	https://documentation.dataspace.copernicus.eu/APIs/SentinelHub/Process/Examples/S2L2A.html	True color [B04, B03, B02]	20 min	True-color composition
R18	Shuttle Radar Topography Mission (SRTM)	NASA Earthdata	https://www.earthdata.nasa.gov/data/instruments/srtm	Mission, coverage, products and limitations	45 min	Source DEM
R19	Python Quickstart	Rasterio	https://rasterio.readthedocs.io/en/stable/quickstart.html	Open/read, metadata, bands, bounds, transform, CRS, write	1.5 h	GeoTIFF and Rasterio
R20	NOAA/NOS's VDatum: A tutorial on datums	NOAA	https://vdatum.noaa.gov/docs/datums.html	Horizontal vs vertical datum concepts	1 h	Vertical datums
R21	Tutorial - User Guide	FastAPI	https://fastapi.tiangolo.com/tutorial/	First Steps, Request Body, Errors, Background Tasks, Static Files, Testing	3 h	REST API and jobs
R22	Request Files	FastAPI	https://fastapi.tiangolo.com/tutorial/request-files/	UploadFile, multipart forms and limits	45 min	File uploads
R23	Models	Pydantic	https://docs.pydantic.dev/latest/concepts/models/	Validated models, fields and serialization	1 h	Job/manifest schema
R24	Quick Start	React	https://react.dev/learn	Components, state, events, conditional rendering, lists	2 h	TerraFly UI
R25	TypeScript for the New Programmer	Microsoft TypeScript	https://www.typescriptlang.org/docs/handbook/typescript-from-scratch.html	JS-to-TS mental model and type checking	1 h	Frontend types
R26	Fundamentals	Three.js	https://threejs.org/manual/en/fundamentals.html	Scene, camera, renderer, mesh, geometry, material, light	1.5 h	3D viewer
R27	Custom BufferGeometry	Three.js	https://threejs.org/manual/en/custom-buffergeometry.html	Positions, indices, normals and UV attributes	1 h	Height-field mesh
R28	glTF Tutorial	Khronos Group	https://github.com/KhronosGroup/glTF-Tutorials/gltfTutorial/	Scenes/nodes, buffers, accessors, meshes, materials, textures	2 h	GLB artifact
R29	Linear Regression, Clearly Explained!!!	StatQuest	https://www.youtube.com/watch?v=7ArmBVF2dCs	0:37-14:15 for fit/R-squared; 25:26 for F-distribution context	28 min	Scale and offset calibration
R30	R-squared, Clearly Explained!!!	StatQuest	https://www.youtube.com/watch?v=2AQKmw14mHM	Entire video	11 min	R-squared gate
R31	Cross-validation: evaluating estimator performance	scikit-learn	https://scikit-learn.org/stable/modules/cross_validation.html	Hold-out principle, validation vs test, CV	1 h	Train/validation/test separation
R32	Common pitfalls and recommended practices	scikit-learn	https://scikit-learn.org/stable/common_pitfalls.html	Inconsistent preprocessing, leakage, randomness	1 h	Leakage and reproducibility
R33	Metrics and scoring: quantifying the quality of predictions	scikit-learn	https://scikit-learn.org/stable/modules/model_evaluation.html	Regression metrics: MAE, RMSE, R2	1 h	Evaluation metrics

ID	Exact title	Owner	URL	Required part	Time	Covers
R34	2023 IEEE GRSS Data Fusion Contest	IEEE GRSS	https://www.grss-ieee.org/community/technical-committees/2023-ieee-grss-data-fusion-contest/	Track 2, terms, data, citation	1 h	Multi-city optical/SAR, masks and nDSM
R35	Model Cards	Hugging Face	https://huggingface.co/docs/hub/model-cards	Metadata, intended uses, limitations, evaluation	45 min	Model governance
R36	Geo Guidelines	Google	https://about.google/brand-resource-center/products-and-services/geo-guidelines/	Attribution and Google Earth restrictions	30 min	Imagery use and attribution
R37	About CC Licenses	Creative Commons	https://creativecommons.org/share-your-work/cclicenses/	BY, SA, NC and ND conditions	30 min	Data/media licensing
R38	Get Started	pytest	https://docs.pytest.org/en/stable/getting-started.html	Discovery, assertions, exceptions and temporary paths	1 h	Backend tests
R39	Getting Started	Vitest	https://vitest.dev/guide/	Writing tests and vitest run	45 min	Frontend tests
R40	Secure Hash Standard	NIST	https://www.nist.gov/publications/secure-hash-standard	Purpose of SHA digests and change detection	30 min	SHA-256 evidence

Verification notes

- Links and titles were checked from the linked publisher/project pages or the video's public record on 2026-08-28.
- YouTube timelines can shift if a publisher replaces a video; recheck before a future semester.
- DFC23 access is conditional on its current terms. The table is a learning path, not permission to redistribute data.
- Google Earth may be used only within applicable guidelines; TerraFly must not use captured Earth output to reconstruct a 3D model.

TerraFly claims ledger

This ledger separates what was demonstrated on the packaged build from what remains conditional. A judge claim is permitted only at the level shown here.

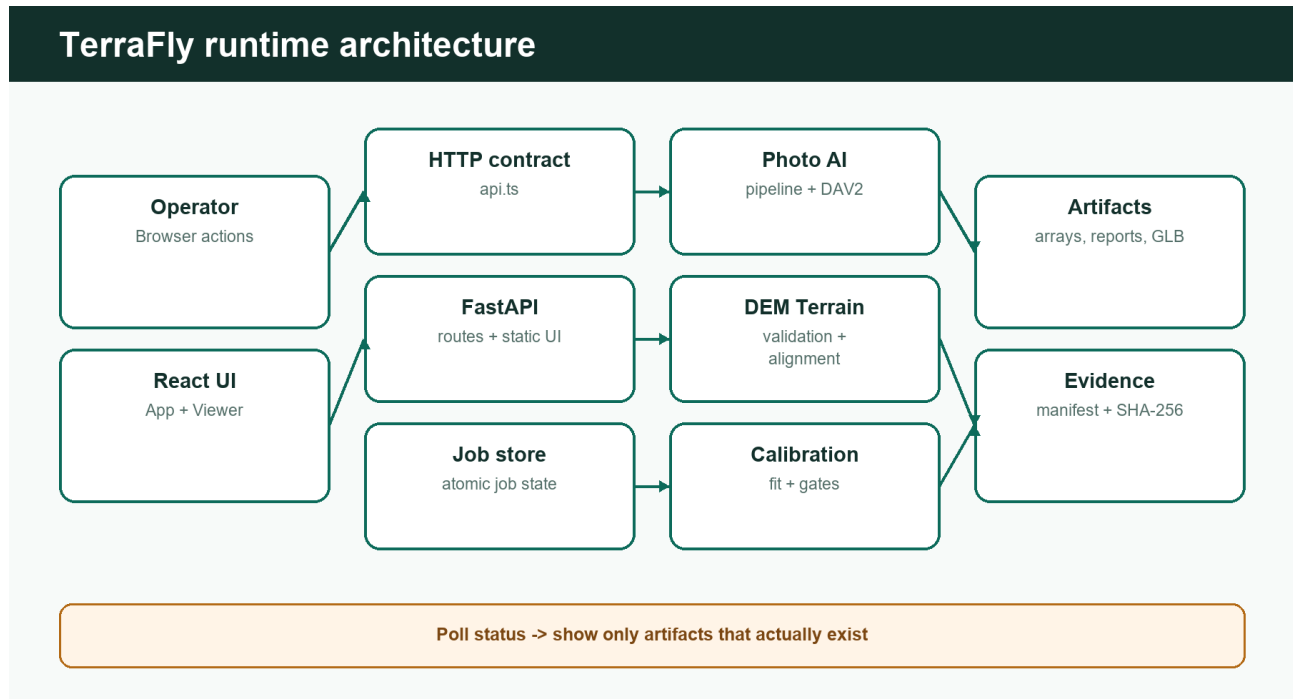
Status	Claim	Evidence	Limitation	Judge wording
PROVED	The packaged backend passes its automated suite.	40 tests passed on 2026-08-28; backend/tests/	Tests use synthetic fixtures for scientific values.	"The packaged backend passes 40 contract, security, terrain, calibration and artifact tests."
PROVED	The packaged frontend builds and passes component/geometry tests.	2 Vitest files, 10 tests; production build, frontend/src/*.test.*	Browser automation cannot prove every GPU/browser combination.	"The interface and measurement geometry pass 10 automated tests and a production build."
PROVED	DEM Terrain preserves supplied metric elevations through alignment and export.	terrain.py:84-192, 213-348; test_terrain.py:65-137; live DEM smoke	Accuracy is inherited from the source DEM; resampling adds no information.	"TerraFly aligns and visualizes a named DEM without claiming that resampling improves its native accuracy."
PROVED	Photo AI uses the real Depth Anything V2 Large checkpoint on CUDA when available.	depth_anything_v2.py:11-158; verified revision 7581137eff8d4e94f6e796d3baea0e9fa79b22d2, device CUDA	It is a general relative-depth model, not an overhead height model.	"Our current AI baseline is the real DAV2 Large checkpoint, recorded by exact revision; its output remains relative until calibrated."
PROVED	Raw output and canonical normalized surface are preserved separately.	pipeline.py:44-76; artifacts.py:735-888; verified raw_model_output.npy and relative_surface.npy	Neither array contains metres by itself.	"We preserve the model field before conversion and a separate normalized 0-to-1 surface for auditability."
PROVED	The historical roof-as-hole direction error is repaired and regression-tested.	conventions.py:9-98; test_adapter_contract.py:24-55; test_artifacts.py:130-166	Correct local ordering does not prove correct absolute height.	"The adapter declares the model convention and normalizes once; a raised-roof regression fails if height is inverted."
PROVED	Tiled inference is bounded, aligns local scale/offset and feather blends.	tiling.py:8-99; test_tiling.py:9-64	Tiling improves memory/detail handling, not model accuracy evidence.	"Large images use bounded overlapping tiles with affine overlap alignment and one final normalization."
PROVED	Metric Photo AI artifacts are locked behind held-out checks.	calibration.py:136-163, 276-532; test_calibration.py:68-241	A global affine fit cannot correct local domain errors; provenance of user evidence still matters.	"Metric export appears only after independent controls and held-out validation satisfy all documented gates."
PROVED	The bundled calibration demo exercises the complete pass path.	verified CUDA calibration smoke: scale about 40, offset about 100, RMSE about 0.000002 m, 18 artifact hashes	The reference is derived from the output and is not real accuracy evidence.	"This is a synthetic software oracle that proves the gate and artifact path, not the model's accuracy."
PROVED	GLB output contains a texture, UVs, normals and lit material.	artifacts.py:83-296; test_artifacts.py:31-128	It is a height field; steep faces lack real facade pixels.	"The export is a portable textured height mesh with computed normals; neutral steep faces avoid inventing facade texture."
PROVED	The viewer supports orbit, first-person, photo/height modes, wireframe, sun and A/B sampling.	SurfaceViewer.tsx:45-440; surfaceGeometry.ts:122-255; live screenshots	First-person has no collision/terrain-following and the mesh is downsampled.	"The browser provides interactive inspection and point comparison, not a flight simulator or survey instrument."
PROVED	Inputs and artifacts are bounded and path-contained.	imaging.py:31-175; jobs.py:20-26; main.py:206-231; test_api.py:81-131	Local-only operation reduces exposure but does not replace a production security review.	"Uploads, job IDs and artifact paths are validated and the service binds locally."
PARTIAL	TerraFly addresses the SIH single-view height goal.	Photo AI relative path plus reference/GCP calibration	No independent multi-landscape Photo AI benchmark; coarse DEM fusion is not implemented as a learned absolute DSM method.	"We implemented the full evidence-aware pipeline and calibration gate; real single-view height accuracy is the next validated milestone."

Stat us	Claim	Evidence	Limitation	Judge wording
PA RTI AL	TerraFly works offline.	Local runtime and prebuilt UI work after setup/cache	First setup and uncached model download require internet; the release omits model weights and environments.	"After one-time dependency and model setup, the demonstrated workflow runs locally without sending scene data to a cloud service."
PA RTI AL	TerraFly supports real mountains.	DEM Terrain accepts licensed, overlapping optical GeoTIFF + metre DEM	The bundled hero pair is synthetic; a real Sentinel-2/SRTM judge pair must be prepared and attributed.	"Mountains are reconstructed from the supplied DEM evidence; today we are showing the software fixture unless the real pair and attribution are visible."
PA RTI AL	The Structures layer makes cities clearer.	artifacts.py:590-732; surfaceGeometry.ts:66-105	Candidates are nonsemantic and may be trees, missed roofs or false positives.	"Structures is an optional visual aid and never changes the scientific surface or measurements."
BL OC KE D	Photo AI RMSE/MAE/correlation on real remote-sensing scenes.	docs/MODEL_METRIC_CARD.md:1-27; evaluator exists in evaluation.py:9-50	No accepted independent aligned RGB-DSM/nDSM benchmark in the release.	"The real metric card is intentionally blocked until licensed held-out truth is acquired."
BL OC KE D	Survey-grade or high-precision height.	no evidence	General monocular model, affine calibration and source DEM limits	Never claim it.
BL OC KE D	Disaster prediction or operational deployment.	no hazard model or operational validation	Terrain is only one input to hazard analysis.	"TerraFly can supply an elevation inspection foundation; hazard prediction needs separate models, data and validation."
BL OC KE D	Google Earth screenshot reconstruction.	docs/JUDGE_QA.md:19-25; Google Geo Guidelines	Screenshot lacks height metadata; Google terms restrict 3D reconstruction.	"We do not use Google Earth imagery to reconstruct TerraFly geometry."
FU TU RE	Overhead-specific building mask, nDSM and uncertainty model.	docs/TRAINING_ROADMAP.md:17-69	Requires licensed aligned data and geographic splits.	"This is the next separately versioned research stage, not a feature in today's build."
FU TU RE	Optional SAR fusion and multi-city domain adaptation.	DFC23 Track 2 plan	Terms, data audit and RGB baseline must pass first.	"SAR will be an ablation retained only if held-out geographic performance improves."

Visual evidence and editable diagrams

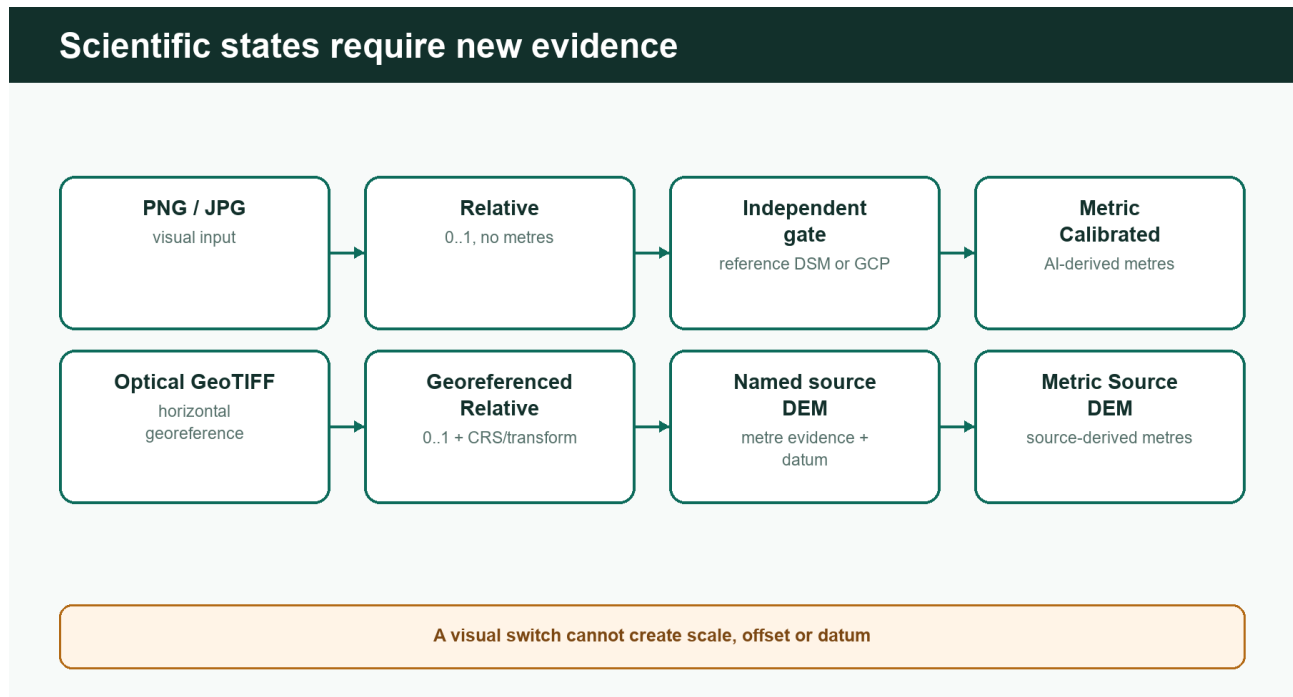
The diagram sources are editable Mermaid files in docs/ownership/diagrams. Screenshot SVG overlays remain editable; annotated PNGs are flattened copies for portable viewing. Original captures are preserved.

TerraFly runtime architecture



Poll status -> show only artifacts that actually exist

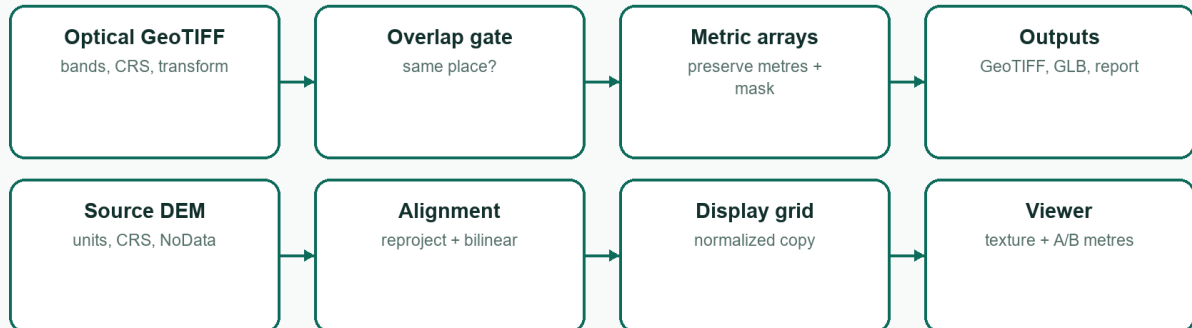
Scientific states require new evidence



A visual switch cannot create scale, offset or datum

DEM Terrain workflow

DEM Terrain workflow

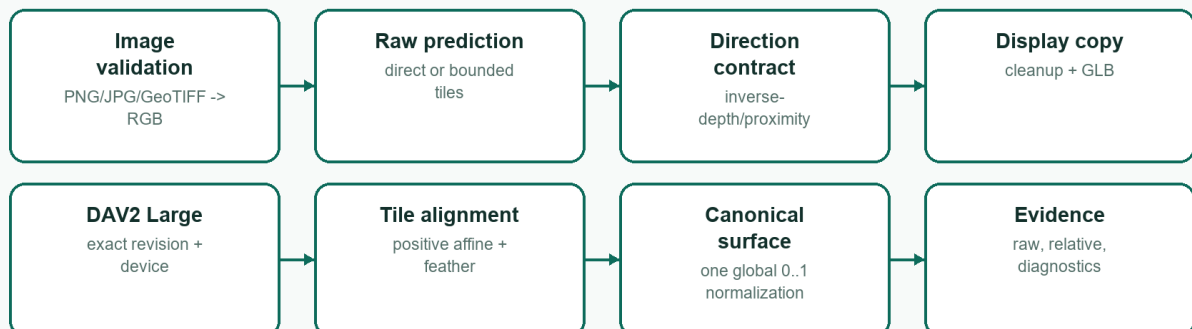


Resampling aligns the grid; it never upgrades native DEM information

Resampling aligns the grid; it never upgrades native DEM information

Photo AI workflow

Photo AI workflow

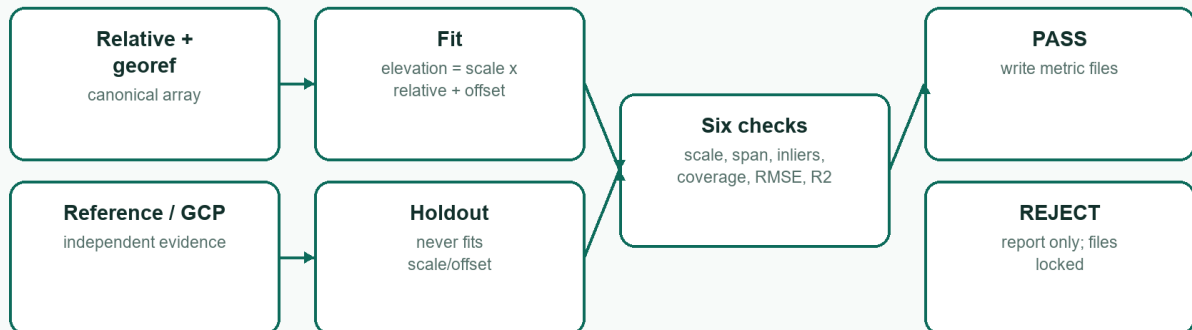


The first output remains relative even when it looks three-dimensional

The first output remains relative even when it looks three-dimensional

Metric calibration gate

Metric calibration gate

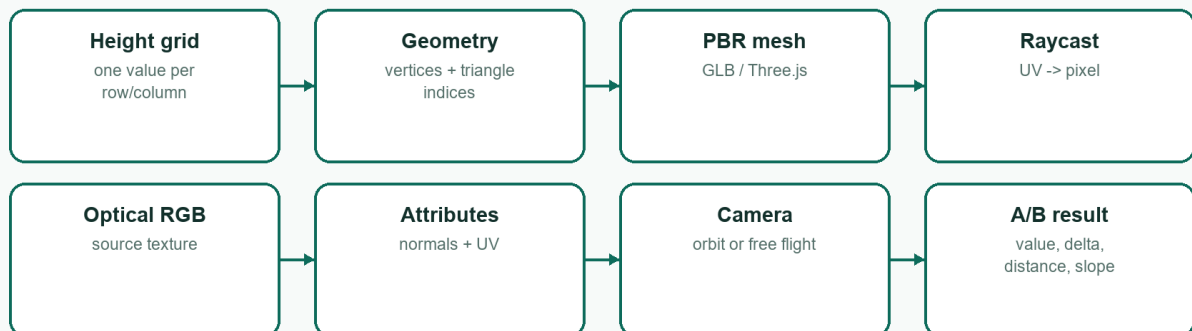


Rejection is a correct scientific result, not a software failure

Rejection is a correct scientific result, not a software failure

Height pixels to textured mesh and measurement

Height pixels to textured mesh and measurement

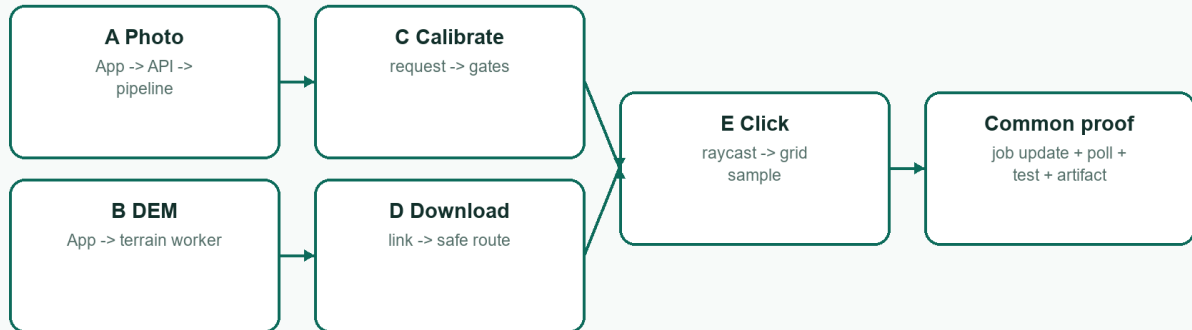


Display exaggeration changes vertices, never stored measurements

Display exaggeration changes vertices, never stored measurements

Five auditable request flows

Five auditable request flows



Every visible result has a route, worker, stored evidence and supporting test

Every visible result has a route, worker, stored evidence and supporting test